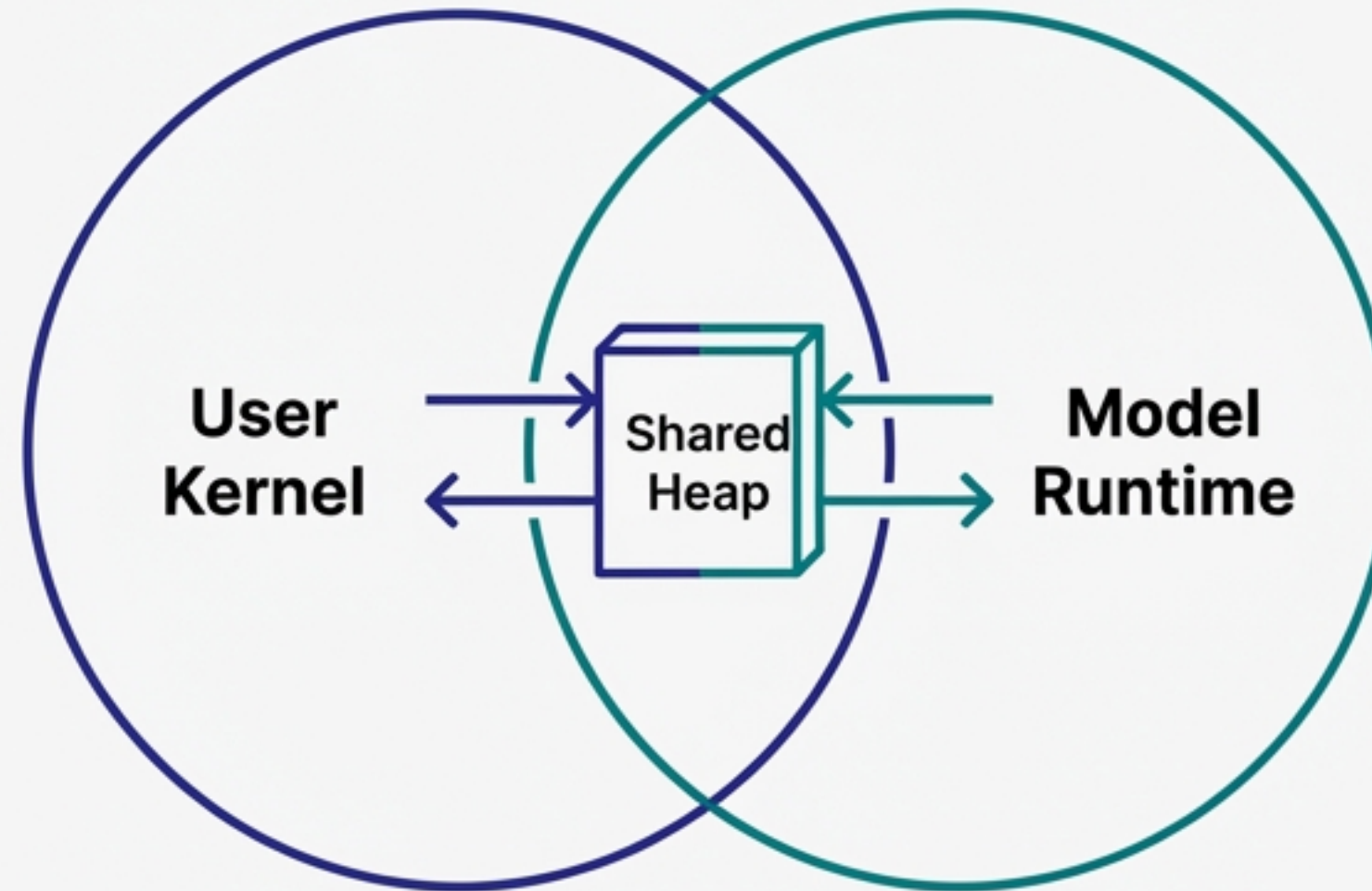


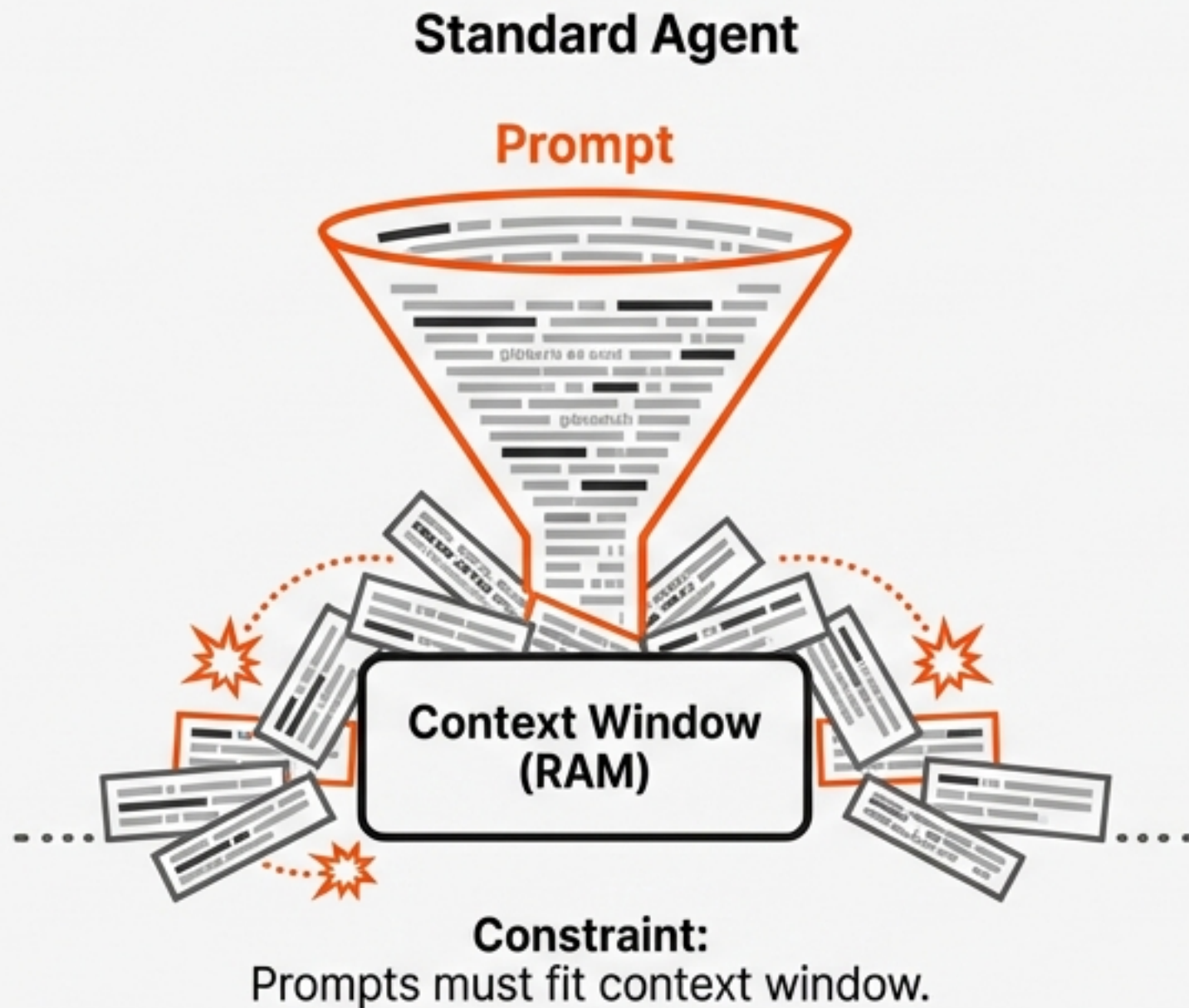
Recursive Language Models in Jupyter

Bringing the Human into the Loop of Self-Orchestrating Agents.

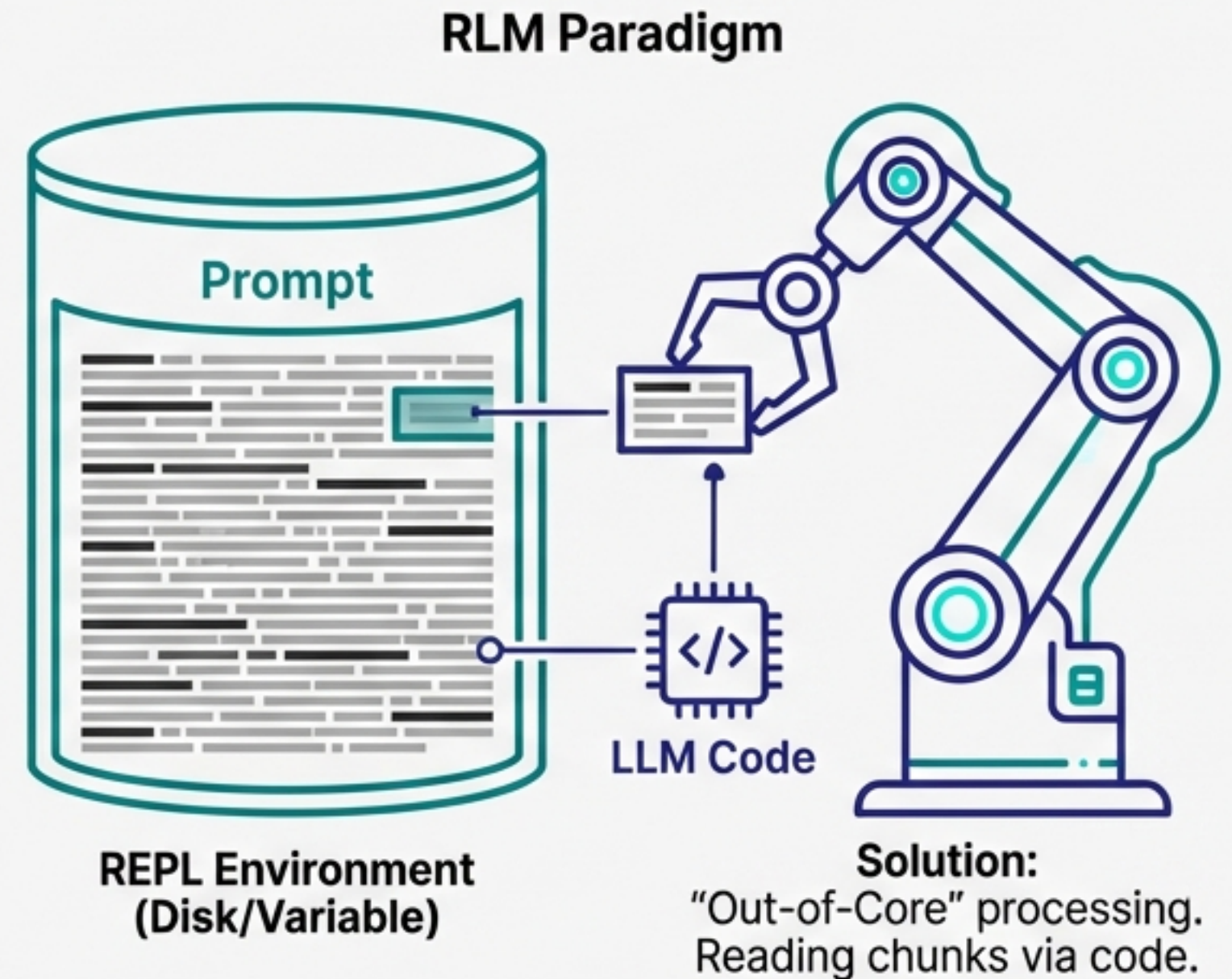


Presented by **Rooted Layers**: AI insights grounded on research.

Treating Prompts as Environmental Variables, Not Inputs

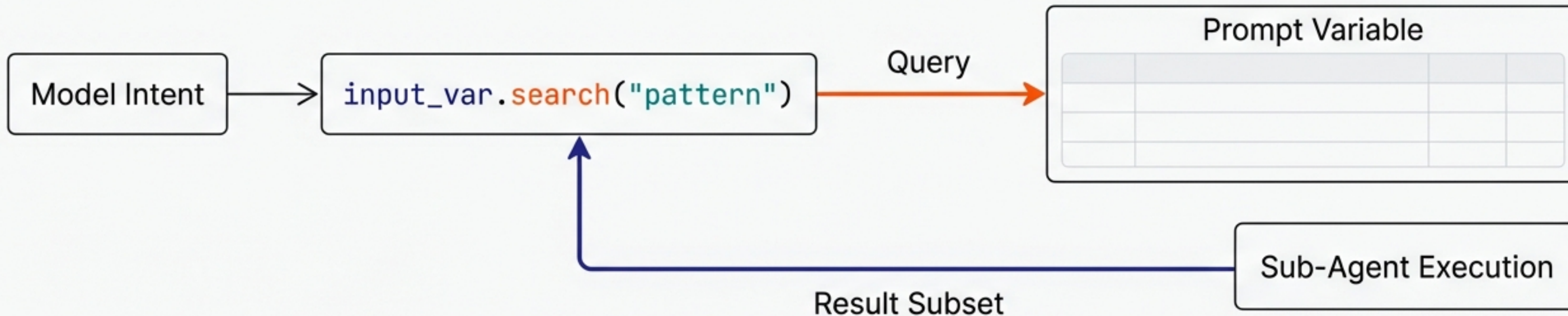


Recursive Language Models (Zhang, Kraska, and Khattab, 2025) introduce a paradigm shift. The prompt is stored as a variable in the REPL environment, not passed directly to the context window.



Just as algorithms process datasets too large for RAM by reading chunks from disk, RLMs process text too large for context windows by reading chunks via code.

How the Model Programs Its Own Thinking



Emergent Patterns



Regex Filtering

Filtering context based on priors (e.g., extracting specific patterns).



Recursive Chunking

Splitting documents and dispatching transformations across those chunks.



Verification

Verifying answers through sub-calls before committing to a final response.

Transcending the Categories of Long-Context Handling

Scaffold-Driven Memory

MemGPT, MemWalker

Structure fixed by engineer. The model navigates a **maze built by humans**.



Code-as-Action

CodeAct

Uses a REPL, but passes input directly to context window. Bottleneck at input size.



Recursive Decomposition

THREAD, ReDel

Allows sub-agents, but assumes input fits in memory.



The RLM Advantage

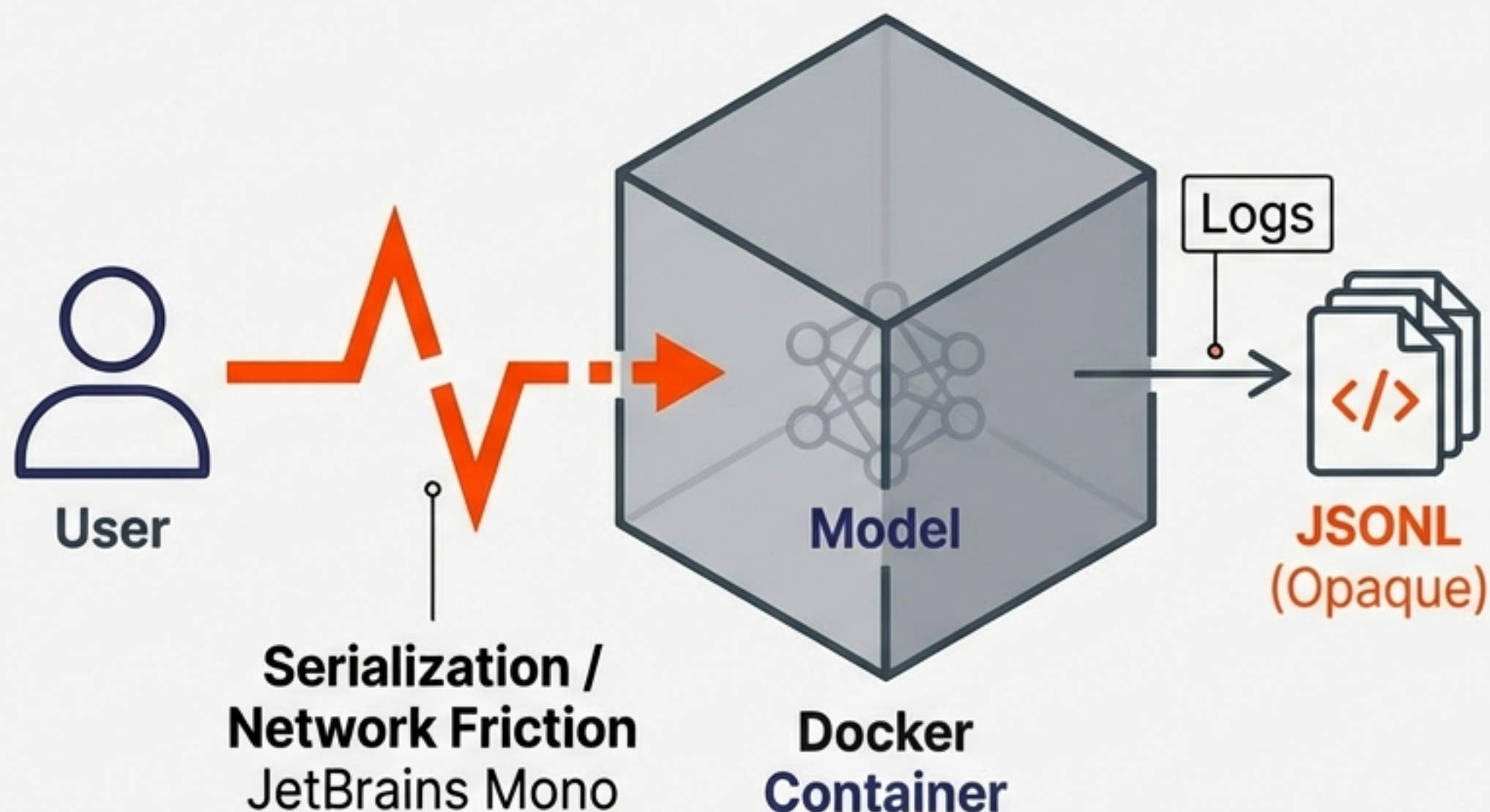
Combines REPL + Recursion + Externalized Prompt.

11 Million Tokens

Processed with 128K-context models (BrowseComp+ Table 1).



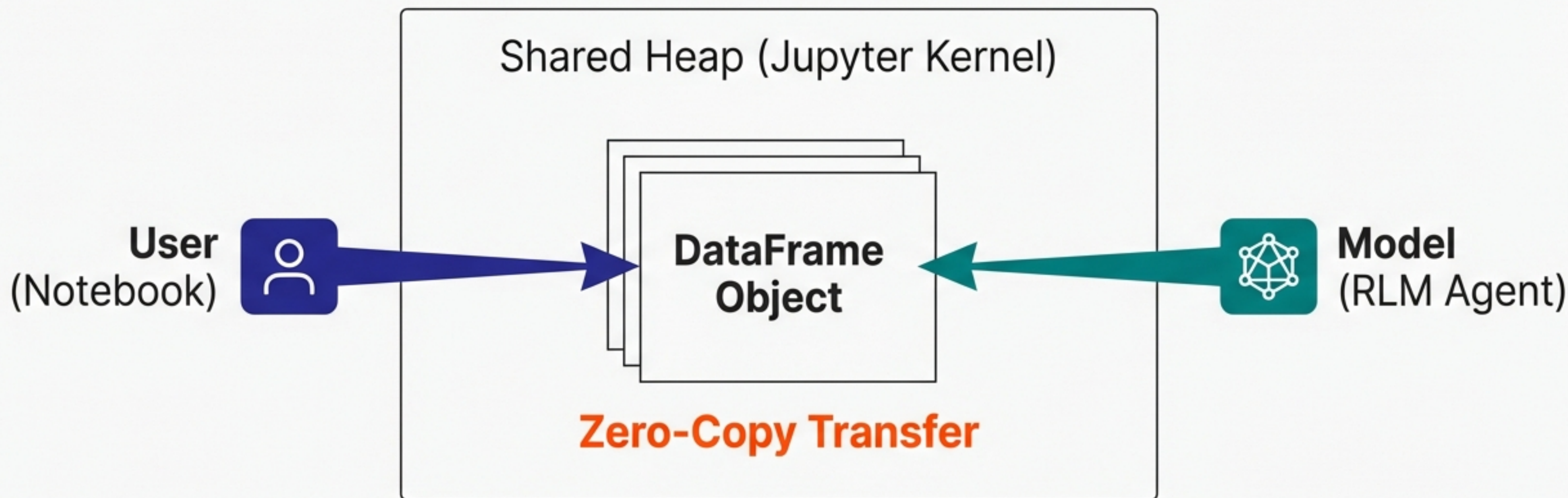
The 'Black Box' Problem in Reference Implementations



-  **The Status Quo:** Reference RLM executes in Docker/Sandboxes. The REPL is isolated.
-  **High Overhead:** Passing DataFrames requires serialization and network transfer.
-  **Invisibility:** Logs are opaque JSONL files.
-  **No Collaboration:** Turn-based prompting, not a shared workspace.

To make this power usable, we must break the wall between the User and the Model.

Breaking the Wall: The Shared Namespace

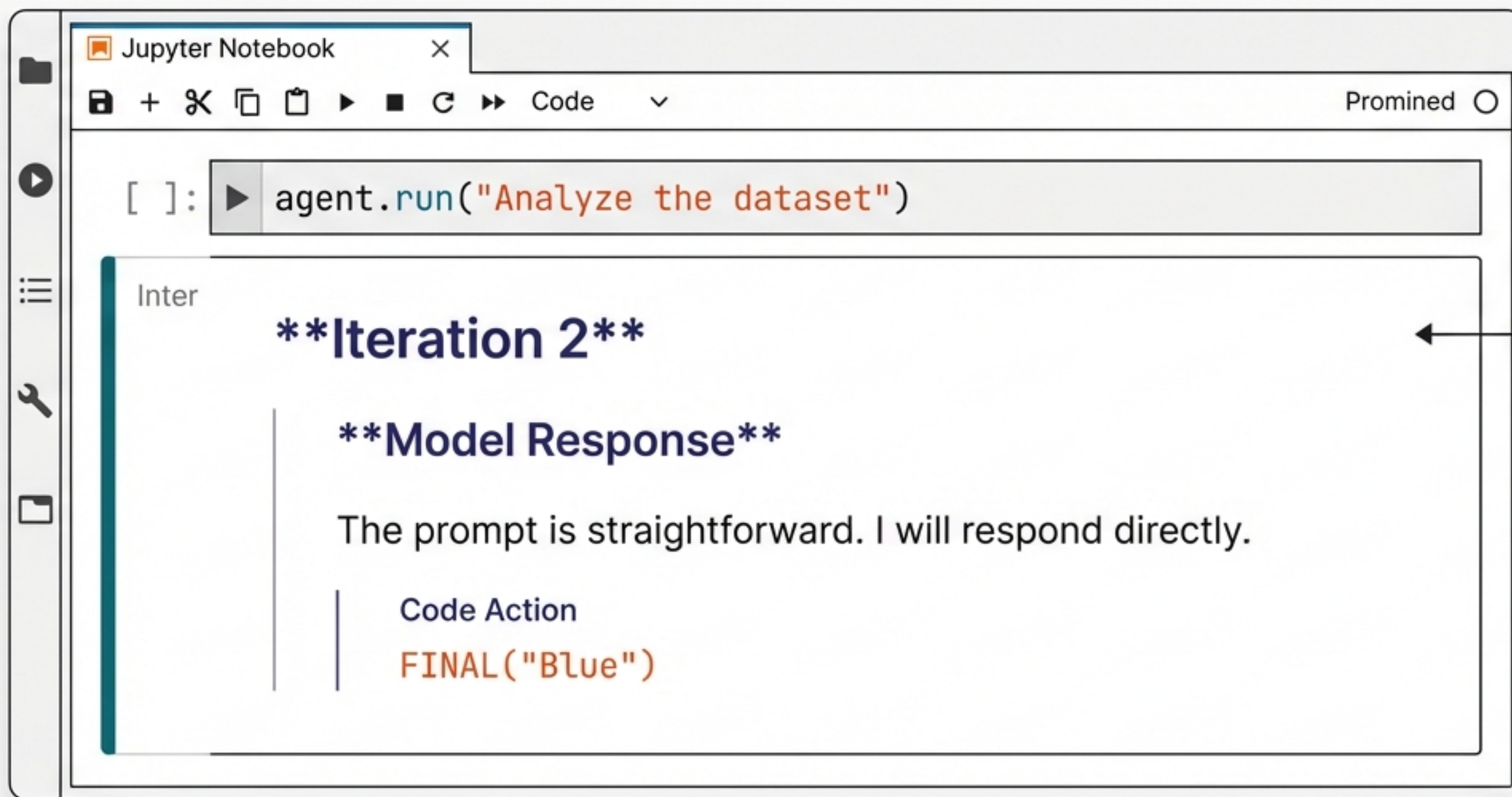


The Innovation: Moving execution from container to User's Kernel.

The Paradigm: "Not turn-based prompting. Pair programming on a shared heap."

Benefit: The model sees your variables; you inspect its artifacts.

Rendering Thought Processes with Trace Artifacts



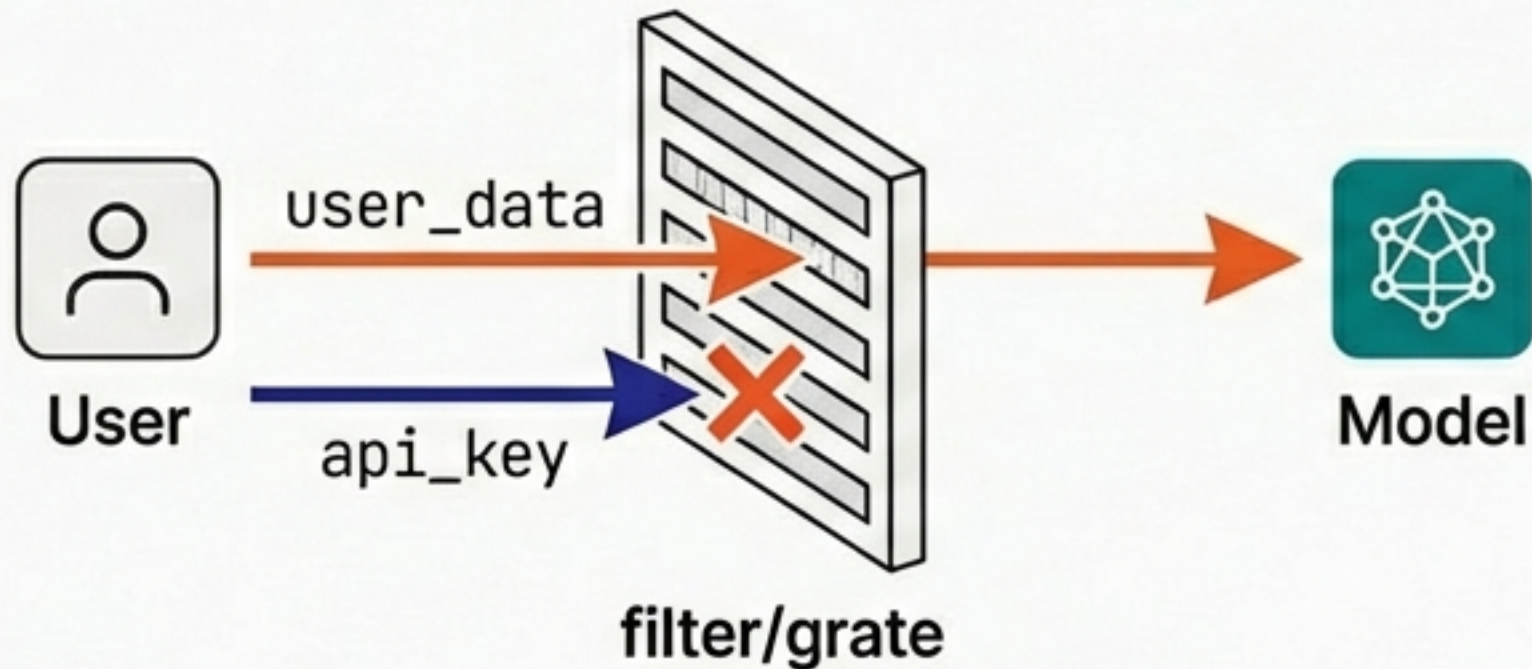
← Inline Markdown traces
replace raw JSONL logs.

Visualize the recursion
loop
(Response → Code
→ Sub-call).

Synchronizing Reality: Managing the Shared State

Inter

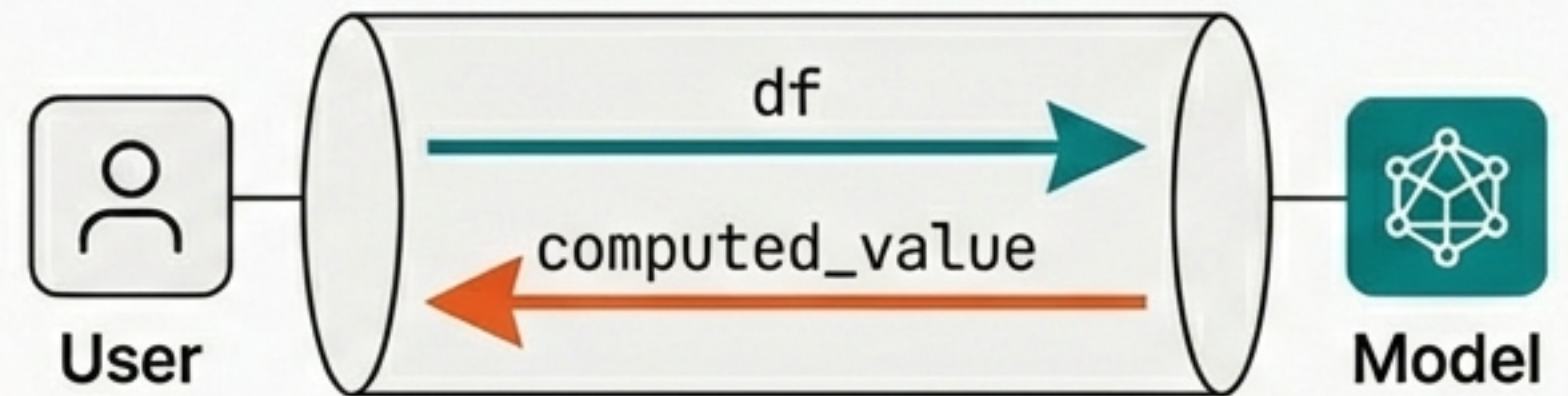
Mode 1: Selective Sync (Allowlist)



Data flows through the filter.

Inter

Mode 2: Full Bidirectional Sync



Data flows freely in both directions.

Inter

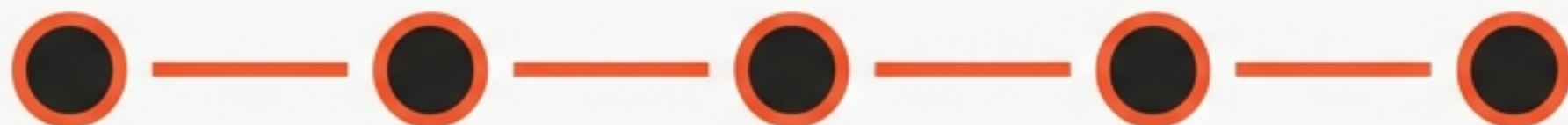
Riskier but powerful. Objects pass by reference.

Inter

Enables workflows where the model creates a variable that persists for user analysis.

Stateless Completions vs. Stateful Sessions

Stateless Completions



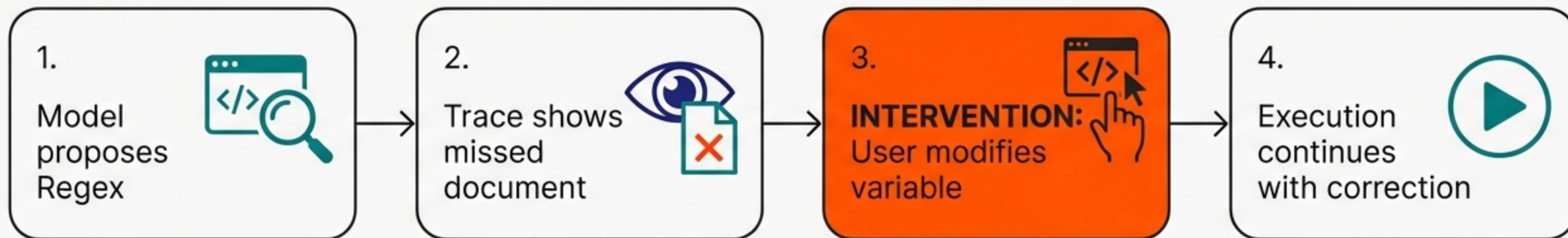
```
# Programmatic Access  
history = session_context_0  
persist = True
```

Stateful Sessions



Completions are independent. Sessions maintain history at the prompt and REPL level. With 'persistent=True', variables defined in call 1 (e.g., 'x') are available in call 2.

Surgical Intervention: The Human as Co-Pilot

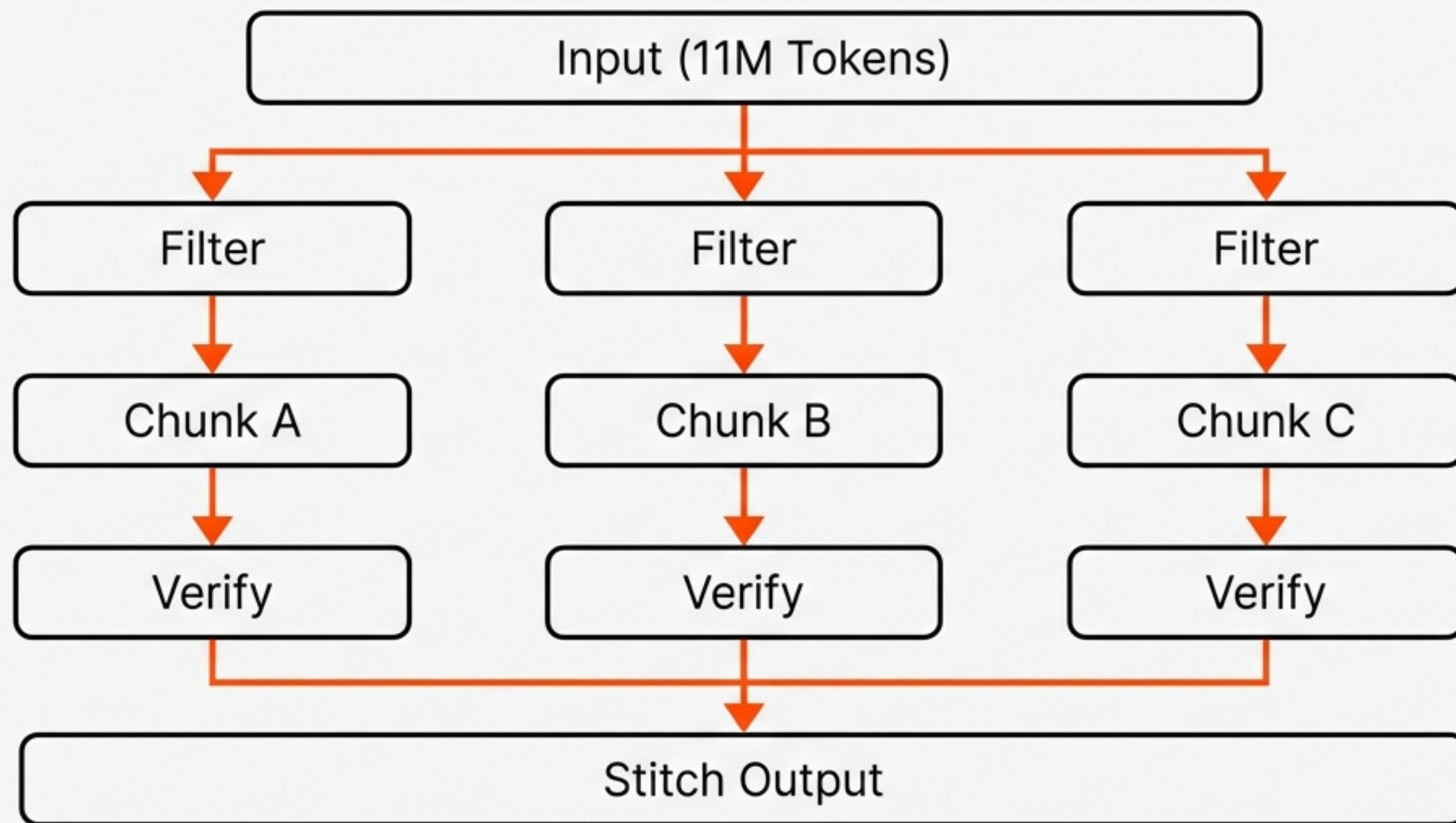


**“The model remains the driver;
you become the co-pilot.”**

Trace artifacts make failure legible. The shared REPL makes failure fixable.

Performance at Scale: 11 Million Tokens

Stitching Trajectory



Key Metrics

Inter, JetBrains Mono

Source:
BrowseComp+
Experiments
(Zhang et al., 2025)

Metric:
**58% F1 on
OOLONG-Pairs**

Comparison:
Base GPT-5 < 0.1%

Trade-offs: Security, Cost, and Latency



Security

Non-Isolated Environment

Malformed code can crash the session. Use Docker for untrusted workloads.



Cost

High Variance

Median costs are low, but percentile costs can reach **3-5x** due to complex recursive trajectories.

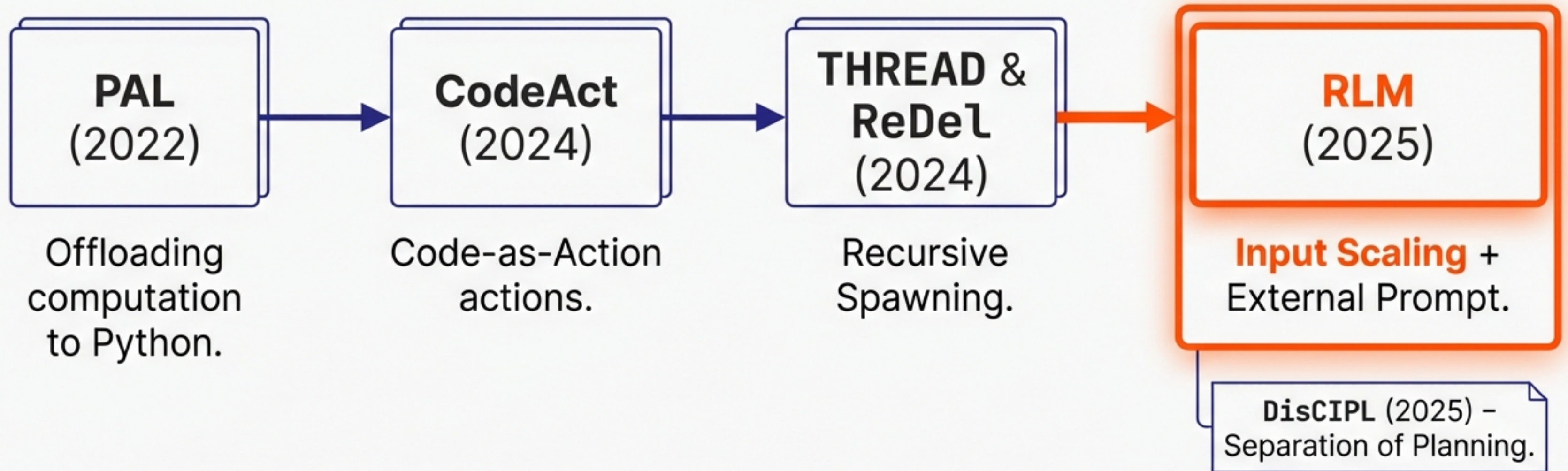


Latency

Sequential Execution

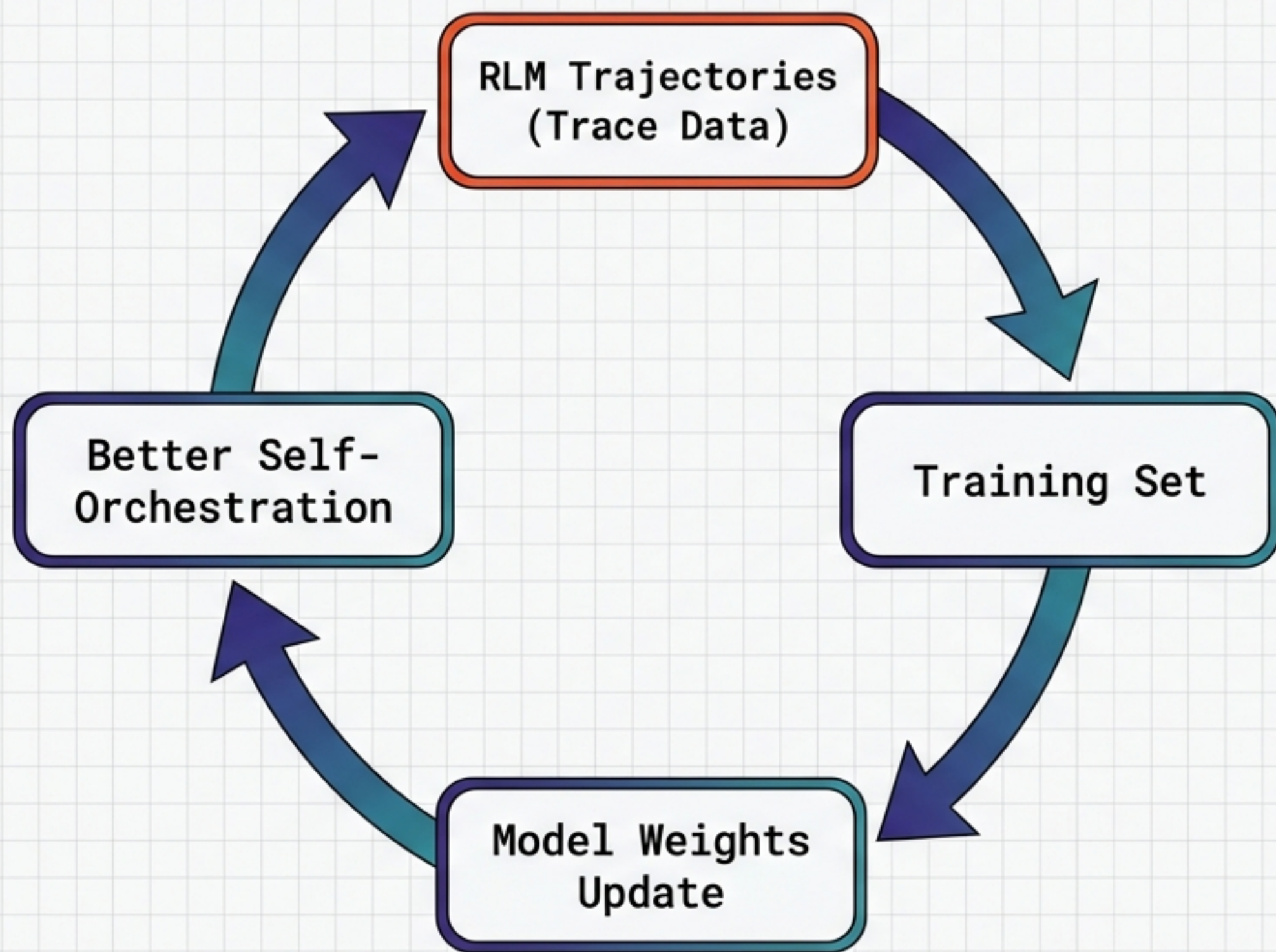
Sub-calls are sequential. Recursion depth currently capped at **1** for manageability.

Grounded in Research: The Lineage of RLM

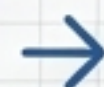


RLM synthesizes CodeAct's REPL and THREAD's recursion, adding **"Out-of-Core"** variable management.

Future Outlook: Bootstrapping Reasoning



Current State: Inference-time scaffolds.



The Horizon: Training models to be RLMs.



Insight: Viewing trajectories not just as logs, but as reasoning data for model evolution.

Implementation and Resources

Call to Action: Try the Fork (PR [#46](#))

- Full Repository Access
- Trace Notebooks (**.trace.ipynb** generation)
- Demos: Sessions, Persistence, Sync Modes

Rooted Layers